

SMART_AO V8 — DATA-01

Mapping de persistance SQLAlchemy et migrations Alembic — Premier slice

Version : 1.0

Statut : contrat de persistance à appliquer avant la création des modèles SQLAlchemy et migrations V8

Auteur : Manus AI

Périmètre : Case , Consultation , DceVersion , Decision , idempotence, événements, outbox et séquenceurs du premier slice

Dépendances normatives : DOMAIN-01 v1.1, DOMAIN-03 v1.0, APP-01 v1.0, TEST-01 v1.0

1. Objet et décision de gel

DATA-01 traduit les aggregates et invariants du premier slice en tables PostgreSQL, modèles SQLAlchemy 2.x et migrations Alembic manuelles. Il ne crée ni écran, ni endpoint, ni logique de décision. Les règles métier restent dans le domaine et les handlers ; la base garantit les identités, le tenant, les relations locales, les unicités, la révision optimiste et la non-destruction structurelle.

Décision DATA-01 : une table n’ est pas créée parce qu’ un écran en a besoin, mais parce qu’ un aggregate possède une réalité durable, une entité interne identifiée, un fait append-only ou une exigence de reprise. Aucune table ne devient un raccourci pour écrire plusieurs aggregates métier dans la même transaction.

Garanties que la persistance doit apporter	Garanties qui restent hors de la base seule
Tenant obligatoire, clés étrangères tenant-scoped, contraintes d’ unicité et index.	Autorisation d’ un acteur, politique patron/collaborateur et droit contextuel.
Révision optimiste d’ un root, immutabilité des corpus DCE et append-only des contextes finalisés.	Compatibilité complète d’ une Case avec un lot, un DCE ou une décision.
Root + Domain Event + Outbox + résultat idempotent atomiques.	Orchestration des effets inter-aggregate par un Process Manager.

Absence de cascade `DELETE` inter-root et conservation de l' historique.

Calcul de readiness, évaluation DCE ou décision humaine.

2. Décisions communes SQLAlchemy et PostgreSQL

2.1. Conventions immuables

Sujet	Convention DATA-01
SGBD	PostgreSQL 16+ ; <code>UUID</code> , <code>TIMESTAMPTZ</code> , <code>JSONB</code> et index partiels PostgreSQL sont autorisés.
Schéma	Un schéma applicatif unique <code>public</code> pour le pilote. Les bounded contexts sont séparés par modules Python et préfixes de tables, pas par microservices.
Identifiants	UUID générés par l' application ; aucune extension PostgreSQL de génération d' <code>UUID</code> n' est requise.
Temps	UTC exclusivement ; colonnes <code>created_at</code> , <code>updated_at</code> , événements et receipts en <code>TIMESTAMPTZ</code> .
Tenant	Chaque table métier, événement, outbox, receipt et séquenceur contient <code>tenant_id</code> <code>NOT NULL</code> . Toute table référencée par une FK composite porte aussi le <code>UNIQUE (tenant_id, id)</code> redondant requis par PostgreSQL.
Révision	Chaque root porte <code>aggregate_revision</code> <code>INTEGER NOT NULL DEFAULT 0</code> . Les entités internes ne portent pas une révision concurrente autonome.
États	<code>VARCHAR</code> + contraintes <code>CHECK</code> nommées. Aucun enum PostgreSQL global dans le pilote : les états évolueront via migrations explicites.

JSONB	Autorisé seulement pour contenu canonique versionné, payload d' événement ou métadonnées bornées. Aucun état métier implicite caché dans JSONB.
Suppression	Aucun <code>ON DELETE CASCADE</code> entre aggregates. Les objets historiques sont archivés, retirés ou supersédés.
Données financières	Aucune colonne de montant, coût, marge, devis ou trésorerie dans ce slice.

2.2. Nommage des objets SQL

Plain Text

Tables	snake_case pluriel, préfixe du contexte si nécessaire
Clés primaires	pk_<table>
Clés étrangères	fk_<table>__<target>__<column>
Unicités	uq_<table>__<colonnes>
Checks	ck_<table>__<règle>
Indices	ix_<table>__<colonnes>
Index partiels	ux_<table>__<règle> # unique index PostgreSQL
Migrations Alembic	YYYYMMDD_NNNN_<objet>.py

Exemple : `uq_dce_versions__tenant_consultation_corpus_hash` ,
`ck_decision_conditions__deadline_or_reason` , `ix_outbox_messages__pending_delivery` .

2.3. Base SQLAlchemy à implémenter

Python

```

from __future__ import annotations

from datetime import datetime
from uuid import UUID

import sqlalchemy as sa
from sqlalchemy.dialects.postgresql import JSONB, UUID as PG_UUID
from sqlalchemy.orm import DeclarativeBase, Mapped, mapped_column

class Base(DeclarativeBase):
    metadata = sa.MetaData(
        naming_convention={

```

```

        "ix": "ix_%(column_0_label)s",
        "uq": "uq_%(table_name)s__%(column_0_name)s",
        "ck": "ck_%(table_name)s__%(constraint_name)s",
        "fk": "fk_%(table_name)s__%(referred_table_name)s__%(column_0_name)s",
        "pk": "pk_%(table_name)s",
    }
)

```

```

class TenantRecord:
    tenant_id: Mapped[UUID] = mapped_column(PG_UUID(as_uuid=True), nullable=False)
    created_at: Mapped[datetime] = mapped_column(
        sa.DateTime(timezone=True), nullable=False, server_default=sa.func.now()
    )
    updated_at: Mapped[datetime] = mapped_column(
        sa.DateTime(timezone=True), nullable=False, server_default=sa.func.now()
    )

```

```

class RevisionedAggregate(TenantRecord):
    aggregate_revision: Mapped[int] = mapped_column(sa.Integer, nullable=False,

```

Le code de repository n' utilise pas un `UPDATE` sans garde. Toute mutation d' un root suit cette forme conceptuelle :

SQL

```

UPDATE cases
SET commercial_stage = :new_stage,
    aggregate_revision = aggregate_revision + 1,
    updated_at = NOW()
WHERE id = :case_id
    AND tenant_id = :tenant_id
    AND aggregate_revision = :expected_revision;

```

Zéro ligne mise à jour signifie `VERSION_CONFLICT` . SQLAlchemy peut déclarer `version_id_col` , mais le handler reste responsable de fournir la révision lue par l' utilisateur et de retourner l' erreur métier normalisée.

3. Carte relationnelle du premier slice

Plain Text

```

tenants
├─ consultations ─< consultation_lots

```

```

|
|      └─< consultation_tranches
|      └─< dce_versions ─< dce_documents
|                        └─< dce_document_classifications
|                        └─< dce_document_issues
|                        └─< dce_source_statements
|
|─ cases ─< case_consultation_links
|      └─< case_dce_applicability_history
|─ decisions ─< decision_contexts ─< decision_context_references
|      └─< decision_conditions
|─ command_receipts
|─ domain_events
|─ outbox_messages
|─ process_inbox

```

Les flèches représentent une **propriété de persistance interne**, non une permission d'écriture inter-aggregate. Par exemple, `cases.applicable_dce_version_id` est une référence versionnée vers `dce_versions` ; son handler ne modifie jamais la `DceVersion`.

Partie I — Socle de durabilité commun

4. Table tenants

Le produit est déployé sur un VPS dédié par client, mais `tenant_id` demeure une garantie P0 de défense en profondeur. La table est minimale dans le premier slice : elle ne remplace pas le futur module ORG ni l'authentification.

Colonne	Type	Règle
<code>id</code>	UUID PK	Identité tenant stable.
<code>slug</code>	<code>VARCHAR(120)</code>	Unique ; réservé aux opérations internes/admin.
<code>lifecycle</code>	<code>VARCHAR(32)</code>	<code>ACTIVE</code> , <code>SUSPENDED</code> , <code>ARCHIVED</code> .
<code>created_at</code> , <code>updated_at</code>	TIMESTAMPTZ	Horodatage technique.

Contrainte / index	Finalité
<code>uq_tenants__slug</code>	Aucun tenant ambigu.

ck_tenants__lifecycle	Valeurs de cycle fermées.
ix_tenants__lifecycle	Vérification de disponibilité au chargement du contexte.

5. Table `command_receipts` — registre d’ idempotence

Cette table est technique mais durable : elle garantit qu’ une même intention ne crée pas deux Cases, DCE ou décisions. Elle ne possède aucun état métier et n’ est pas un aggregate business.

Colonne	Type	Règle
id	UUID PK	Identifiant interne du receipt.
tenant_id , actor_id	UUID	Tenant obligatoire ; acteur sans FK vers un futur module auth.
command_id	UUID	Unique par tenant ; traçabilité APP-01.
command_type	VARCHAR(120)	Nom fermé côté application.
idempotency_key	UUID	Clé de rejouabilité de l’ intention.
request_hash	CHAR(64)	SHA-256 du contenu canonique commandé.
correlation_id	UUID nullable	Relie parcours et Process Managers.
status	VARCHAR(32)	PROCESSING , SUCCEEDED , REJECTED , FAILED_RETRYABLE , EXPIRED .
lease_expires_at	TIMESTAMPTZ nullable	Reprise contrôlée après interruption.
aggregate_refs_json	JSONB	Références de résultat seulement ; jamais objet riche.
http_status , result_code	INTEGER / VARCHAR(120)	Réponse mémorisée.
response_body_json	JSONB nullable	Corps rejouable, préalablement nettoyé de secrets/données privées.

event_ids_json	JSONB	UUID des événements acceptés ; non source de vérité.
completed_at , created_at , updated_at	TIMESTAMPTZ	Audit et conservation.

Contrainte / index	Finalité
uq_command_receipts__tenant_actor_type_key SUR (tenant_id, actor_id, command_type, idempotency_key)	Contrat d' idempotence V8.
uq_command_receipts__tenant_command_id SUR (tenant_id, command_id)	Une commande traçable une seule fois.
ck_command_receipts__status	États de receipt fermés.
ix_command_receipts__lease_recovery SUR (status, lease_expires_at)	Recherche des traitements interrompus.
ix_command_receipts__correlation SUR (tenant_id, correlation_id)	Diagnostic et RYOW.

6. Tables domain_events , outbox_messages et process_inbox

Table	Colonnes centrales	Contraintes et index	Rôle
domain_events	id , tenant_id , aggregate_type , aggregate_id , aggregate_revision , event_type , payload_version , payload_json , actor_id , command_id , correlation_id , causation_id , occurred_at .	uq_domain_events__tenant_id ; index (tenant_id, aggregate_type, aggregate_id, occurred_at) et (correlation_id) .	Journal des changements métier acceptés ; payload minimal et sans montant.

outbox_messages	id , tenant_id , event_id , topic , payload_version , payload_json , status , attempt_count , next_attempt_at , published_at , dedupe_key .	FK vers domain_events ; uq_outbox_messages__event_topic ; index partiel pending/retry.	Livraison après commit à un projecteur, Process Manager ou intégration future.
process_inbox	id , tenant_id , process_name , event_id , correlation_id , status , attempt_count , last_error_code , created_at , completed_at .	uq_process_inbox__process_event ; index état/retry.	Déduplique un événement reçu par un Process Manager et mémorise la reprise.

Règle transactionnelle non négociable : le handler insère dans la même transaction le root modifié, le ou les domain_events , les outbox_messages associés et le command_receipt terminal. Le projecteur et le Process Manager ne tournent qu’ après commit.

Partie II — Mapping DCE/Consultation et DCE/DceVersion

7. consultations , consultation_lots , consultation_tranches

7.1. Root consultations

Colonne	Type	Mapping / sens
id , tenant_id	UUID PK, UUID NOT NULL	Identité root et isolation. UNIQUE (tenant_id, id) est ajouté pour FKs composites ; la même règle s’ applique à tous les roots et entités internes adressés par (tenant_id, id) .
aggregate_revision	INTEGER	Révision de la Consultation.

functional_identity_hash	CHAR(64)	Hash canonique de ConsultationKey ; toujours présent, y compris fallback sans référence externe.
buyer_legal_name	VARCHAR(240)	Valeur source affichable.
buyer_normalized_id	VARCHAR(120) nullable	Identité normalisée si connue.
external_reference	VARCHAR(240) nullable	Référence acheteur si connue.
object_label , location_label	VARCHAR(240) / VARCHAR(500)	Objet et lieu initiaux.
source_channel , source_reference , source_received_at	VARCHAR / VARCHAR / TIMESTAMPTZ	Provenance initiale.
lifecycle	VARCHAR(32)	OPEN , CLOSED , ARCHIVED .
freshness	VARCHAR(32)	UNKNOWN , CURRENT , REVIEW_REQUIRED .
metadata_history_json	JSONB	Historique de corrections de métadonnées, borné et append-only dans le handler.
audit	created_by_actor_id , updated_by_actor_id , timestamps	Sans FK vers auth.

Contrainte / index	Justification
uq_consultations__tenant_functional_identity	CONS-INV-02 , y compris lorsque buyer/reference sont incomplets.
index unique partiel (tenant_id, buyer_normalized_id, external_reference) si les deux ne sont pas nuls	Défense complémentaire contre doublon normalisé.
ck_consultations__lifecycle , ck_consultations__freshness	États fermés.
index (tenant_id, lifecycle, updated_at DESC)	Liste portefeuille / consultation.

7.2. Entités internes consultation_lots et consultation_tranches

Table	Colonnes propres	Contraintes
consultation_lots	id , tenant_id , consultation_id , lot_number , label , source_reference , timestamps.	FK composite tenant/consultation ; uq (tenant_id, consultation_id, lot_number) ; aucun FK vers Case.
consultation_tranches	id , tenant_id , consultation_id , tranche_reference , tranche_kind , label , source_reference , timestamps.	FK composite ; uq (tenant_id, consultation_id, tranche_reference) ; check kind.

La suppression d' un lot ou d' une tranche n' est pas une opération de pilote. Une correction est conservée comme métadonnée/provenance ; le numéro source original n' est pas réécrit silencieusement.

8. Root dce_versions

Colonne	Type	Mapping / sens
id , tenant_id , aggregate_revision	UUID / UUID / INTEGER	Root versionné, tenant-scoped.
consultation_id	UUID	FK composite vers Consultation du même tenant.
corpus_hash	CHAR(64)	Empreinte canonique des originaux admis ; immutable après admission.
predecessor_dce_version_id	UUID nullable	FK composite self-reference tenant-scoped ; rectificatif, pas remplacement.
provenance_channel , provenance_reference , provenance_url , source_received_at	VARCHAR / VARCHAR / TEXT / TIMESTAMPTZ	Origine du corpus.
lifecycle	VARCHAR(32)	ADMITTED , SUPERSEDED , WITHDRAWN .
integrity	VARCHAR(32)	VERIFIED , PARTIAL , UNUSABLE .

classification_readiness	VARCHAR(32)	UNCLASSIFIED , PARTIALLY_CLASSIFIED , CLASSIFIED .
analysis_readiness	VARCHAR(32)	NOT_READY , READY_FOR_ANALYSIS , REVIEW_REQUIRED .
withdrawal_source , withdrawal_reason	TEXT nullable	Obligatoires seulement en WITHDRAWN .
superseded_at , withdrawn_at	TIMESTAMPTZ nullable	Audit de cycle.
timestamps / actor audit	standard	Création et dernière mutation root.

Contrainte / index	Justification
uq_dce_versions__tenant_id sur (tenant_id, id)	Cible explicite des FKs composites depuis documents, Case et références internes.
uq_dce_versions__tenant_consultation_corpus_has h	DCE-INV-05 et déduplication corpus.
FK (tenant_id, consultation_id) → consultations(tenant_id, id)	Aucun DCE inter-tenant.
FK (tenant_id, predecessor_dce_version_id) → dce_versions(tenant_id, id)	Rectificatif seulement dans le même tenant.
checks lifecycle/integrity/readiness	DOMAIN-03 § 5.1.
ck_dce_versions__withdrawal_source_when_withdr awn	Retrait prouvé et motivé.
index (tenant_id, consultation_id, source_received_at DESC)	Dernières versions DCE d' une consultation.
index (tenant_id, predecessor_dce_version_id)	Chaîne de rectificatifs.

9. Entités internes de DceVersion

Table	Colonnes	Contraintes et comportement
-------	----------	-----------------------------

dce_documents	id , tenant_id , dce_version_id , storage_object_id , storage_key , original_filename , media_type , byte_size , sha256 , received_from , timestamps.	FK composite root ; uq (tenant_id, id) pour ses enfants et uq (tenant_id, dce_version_id, sha256) ; byte_size > 0 ; storage_object_id , hash et données d' original immuables après admission.
dce_document_classifications	id , tenant_id , dce_document_id , classification , rationale , source , previous_classification_id , is_current , actor/time.	FK composite ; index unique partiel une classification courante par document ; corrections append-only, ancien is_current=false .
dce_document_issues	id , tenant_id , dce_version_id , dce_document_id , issue_kind , impact , locator_json , reason , actor/time.	FK vers root/document tenant-scoped ; checks issue/impact ; l' ajout requiert l' incrément de revision root. Cette table porte un constat métier de fiabilité ; les logs bruts de parsing/OCR restent dans ENGINE/INFRA .
dce_missing_document_declarations	id , tenant_id , dce_version_id , expected_document_family , expectation_source_kind , expectation_source_id , reason , actor/time.	FKs root ; source obligatoire applicativement ; append-only.
dce_source_statements	id , tenant_id , dce_version_id , dce_document_id , locator_json , excerpt , source_language , extraction_origin , actor/time.	FKs root/document tenant-scoped ; locator et extrait obligatoires ; ne contient jamais Requirement , conformités ou interprétation.

9.1. Immutabilité DCE : politique d' application et garde SQL

Les documents de corpus ne sont jamais mis à jour en place. dce_documents accepte uniquement des annotations dans ses tables enfants ; dce_versions.corpus_hash ne change jamais. DATA-01 impose deux protections complémentaires :

1. les repositories n' exposent aucune méthode update_original , delete_document ou replace_corpus ;
2. la migration installe un trigger PostgreSQL qui refuse toute modification des colonnes de contenu d' un corpus admis.

SQL

```
CREATE FUNCTION protect_admitted_dce_content() RETURNS trigger AS $$
BEGIN
    IF OLD.corpus_hash IS DISTINCT FROM NEW.corpus_hash
        OR OLD.consultation_id IS DISTINCT FROM NEW.consultation_id THEN
        RAISE EXCEPTION 'DOCUMENT_ORIGINAL_IMMUTABLE';
    END IF;
    RETURN NEW;
END;
$$ LANGUAGE plpgsql;

CREATE TRIGGER trg_dce_versions_immutable_content
BEFORE UPDATE ON dce_versions
FOR EACH ROW EXECUTE FUNCTION protect_admitted_dce_content();
```

Un second trigger analogue protège `storage_object_id` , `storage_key` , `original_filename` , `media_type` , `byte_size` et `sha256` de `dce_documents` . Les états, readiness et annotations restent modifiables via le root `DceVersion` et incrémentent sa révision.

Partie III — Mapping AFF/Case

10. Root cases

Colonne	Type	Mapping / sens
<code>id</code> , <code>tenant_id</code> , <code>aggregate_revision</code>	UUID / UUID / INTEGER	Root Case .
<code>functional_identity_hash</code>	CHAR(64)	CaseKey canonique : consultation, scope normalisé et origine.
<code>title</code> , <code>object_description</code>	VARCHAR(240) / TEXT	Désignation métier.
<code>business_origin</code> , <code>origin_reference_id</code> , <code>origin_rationale</code>	VARCHAR / UUID nullable / TEXT nullable	Origine Opportunity, manuel, import ou demande client.
<code>consultation_id</code>	UUID nullable	Référence actuelle de Consultation ; nullable seulement pour Case manuelle justifiée.

scope_kind , scope_json , scope_fingerprint	VARCHAR / JSONB / CHAR(64)	CaseScope explicite et stable.
applicable_dce_version_id	UUID nullable	Référence actuelle, jamais possession du DCE.
lifecycle	VARCHAR	ACTIVE , STOPPED , ARCHIVED .
commercial_stage	VARCHAR	États DOMAIN-03 Case § 3.2.
decision_readiness	VARCHAR	NOT_ASSESSED , NOT_READY , READY_WITH_UNKNOWNNS , READY .
dce_freshness	VARCHAR	NO_DCE , CURRENT , REVIEW_REQUIRED .
responsibility_status	VARCHAR	UNASSIGNED , ASSIGNED , ASSIGNMENT_REVIEW_REQUIRED ; projection/commande aval seulement.
stopped_reason , stopped_at , archived_reason , archived_at	TEXT/TIMESTAMPTZ nullable	Motifs non destructifs.
actor/timestamps	standard	Audit de dernière mutation.

Contrainte / index	Justification
uq_cases__tenant_id sur (tenant_id, id)	Cible explicite des FKs composites depuis Decision et historiques Case.
FK composite (tenant_id, consultation_id) et (tenant_id, applicable_dce_version_id)	CASE-INV-03 : références même tenant. La compatibilité consultation/DCE est vérifiée par handler.
ux_cases__tenant_active_functional_identity partiel sur (tenant_id, functional_identity_hash) WHERE lifecycle <> 'ARCHIVED'	CONC-04 : une seule affaire active fonctionnellement identique.
checks lifecycle/stage/readiness/freshness/responsibility	Axes orthogonaux fermés.
check consultation_id IS NOT NULL OR business_origin = 'MANUAL'	Case sans consultation uniquement explicitement manuelle.

index (tenant_id, lifecycle, commercial_stage, updated_at DESC)	Portefeuille et Cockpit.
index (tenant_id, applicable_dce_version_id)	Traitement de rectificatif.

11. Entités internes et historique Case

Table	Colonnes	Contrat
case_consultation_links	id , tenant_id , case_id , consultation_id , scope_snapshot_json , rationale , is_current , actor/timestamps.	Append-only ; index unique partiel une liaison courante par Case ; root porte la référence courante pour lecture rapide.
case_dce_applicability_history	id , tenant_id , case_id , dce_version_id , reason , is_current , set_by_actor_id , set_at .	Append-only ; index unique partiel une version applicable actuelle ; aucune déduction silencieuse lors d' un rectificatif.
case_partner_approvals	Hors migration initiale sauf besoin direct de Case.	Entité DOMAIN-01 conservée mais non implémentée dans le premier démonstrateur.

cases ne possède aucune FK vers Task, Decision, Pricing, Submission ou preuve. Une Decision peut référencer la Case, mais Case ne charge pas une collection riche de Decisions ORM.

Partie IV — Mapping DEC/Decision

12. Root decisions

Colonne	Type	Mapping / sens
id , tenant_id , aggregate_revision	UUID / UUID / INTEGER	Root Decision .
decision_type	VARCHAR	GO_NO_GO , RISK_ACCEPTANCE , PARTNER_SELECTION ,

		PRICING_APPROVAL , SUBMISSION_AUTHORIZATION .
subject_type , subject_id , case_id , scope_fingerprint	VARCHAR / UUID / UUID / CHAR(64)	Sujet explicite et Case/scope de contexte.
decision_key_hash , cycle_number	CHAR(64) / INTEGER	Identité fonctionnelle et cycle de supersession.
lifecycle , outcome , validity , condition_status , context_status	VARCHAR	Cinq axes DOMAIN-03 § 6.2, jamais fusionnés.
selected_final_context_id	UUID nullable	Context figé retenu pour une finalisation. Après création de decision_contexts , une FK composite (tenant_id, id, selected_final_context_id) → decision_contexts(tenant_id, decision_id, id) garantit qu' il appartient bien à cette Decision.
successor_decision_id	UUID nullable	Lien à la nouvelle décision finale après supersession.
final_justification	TEXT nullable	Justification patron de l' outcome.
finalized_by_actor_id , finalized_at	UUID / TIMESTAMPTZ nullable	Signature métier patron/délégataire.
review_required_reason , review_required_at	TEXT / TIMESTAMPTZ nullable	Staleness sans réécriture outcome.
cancel_reason , cancelled_at	TEXT / TIMESTAMPTZ nullable	Annulation d' un brouillon uniquement.
actor/timestamps	standard	Audit de cycle.

Contrainte / index	Justification
uq_decisions__tenant_id sur (tenant_id, id)	Cible explicite des FKs composites depuis contexts et conditions.
FK (tenant_id, case_id) → cases(tenant_id, id)	Décision même tenant / Case.
FK composite finale (tenant_id, id, selected_final_context_id) →	Le contexte final sélectionné appartient au root Decision concerné, sans cycle de

decision_contexts(tenant_id, decision_id, id) ajoutée après création des enfants	création SQL insoluble.
FK self-reference (tenant_id, successor_decision_id)	Supersession même tenant.
uq_decisions__tenant_key_cycle sur (tenant_id, decision_key_hash, cycle_number)	Cycle explicite et historique conservé.
checks pour chaque axe	Valeurs fermées DOMAIN-03.
check finalisation	Si lifecycle='FINALIZED' , alors outcome ≠ UNDECIDED , selected_final_context_id , finalized_by_actor_id , finalized_at et justification non nuls.
check go conditionnel	Si outcome CONDITIONAL_GO , condition_status <> NOT_APPLICABLE ; présence détaillée de condition vérifiée par handler et test DB de fin de transaction.
index (tenant_id, case_id, decision_type, lifecycle)	Dossier de décision.
index (tenant_id, validity, updated_at DESC)	Décisions à revoir.

13. Entités internes Decision

Table	Colonnes	Contraintes / comportement
decision_contexts	id , tenant_id , decision_id , sequence_number , context_fingerprint , canonical_context_json , rationale , unknowns_json , prepared_at , context_state , is_selected_final , actor/timestamps.	FK composite root ; uq (tenant_id, id) et uq (tenant_id, decision_id, id) pour références composites ; uq (tenant_id, decision_id, sequence_number) ; index unique partiel un is_selected_final=true par Decision ; contenu non modifiable après FROZEN /sélection finale.
decision_context_references	id , tenant_id , decision_context_id , aggregate_type , aggregate_id , aggregate_revision , content_hash , reference_role .	FK composite context ; index (tenant_id, aggregate_type, aggregate_id) ; références polymorphes validées tenant-side par handler.

decision_conditions	id , tenant_id , decision_id , label , owner_actor_id , due_at , due_date_absence_reason , failure_consequence , status , satisfied_evidence_ref_json , failure_reason , waiver_justification , timestamps.	FK composite root ; check due_at IS NOT NULL OR due_date_absence_reason IS NOT NULL ; check consequence non vide ; état OPEN/SATISFIED/FAILED/WAIVED .
---------------------	---	---

13.1. Garde d’ immutabilité de DecisionContext final

Le handler ne met jamais à jour canonical_context_json , context_fingerprint , références ou rationale d’ un context FROZEN sélectionné. Une actualisation crée un nouveau context avec sequence_number + 1 . La migration ajoute un trigger qui refuse le changement de contenu lorsque l’ ancien context est figé ou sélectionné final.

```

SQL

CREATE FUNCTION protect_frozen_decision_context() RETURNS trigger AS $$
BEGIN
    IF OLD.context_state = 'FROZEN' OR OLD.is_selected_final THEN
        IF OLD.context_fingerprint IS DISTINCT FROM NEW.context_fingerprint
            OR OLD.canonical_context_json IS DISTINCT FROM NEW.canonical_context_json
            OR OLD.rationale IS DISTINCT FROM NEW.rationale THEN
            RAISE EXCEPTION 'DECISION_CONTEXT_IMMUTABLE';
        END IF;
    END IF;
    RETURN NEW;
END;
$$ LANGUAGE plpgsql;

```

Partie V — Propriétés SQLAlchemy et interdits ORM

14. Mapping SQLAlchemy par root

Root	Module Python cible	relationship() autorisées	Interdits
------	---------------------	------------------------------	-----------

ConsultationRecord	app/modules/dce/infrastructure/models/consultation.py	Lots/tranches internes, cascade="all, delete-orphan" uniquement pour enfants non historiques explicitement autorisés.	Relation de collection Case ou DceVersion avec cascade.
DceVersionRecord	app/modules/dce/infrastructure/models/dce_version.py	Documents, classifications, issues, source statements ; aucune cascade hard delete en production.	relationship(CaseRecord) , relationship(DecisionRecord) , suppression de document/original.
CaseRecord	app/modules/case/infrastructure/models/case.py	Liens/historique Case internes sans cascade delete.	Collections Decision, DceVersion, Task, Pricing, Submission ou preuve.
DecisionRecord	app/modules/decision/infrastructure/models/decision.py	Contexts et conditions internes ; contexts append-only.	Relation Case chargée comme objet mutable, collections Task/Action/Pricing/Submission.

Même lorsqu' une relation interne est techniquement configurée avec delete-orphan , l' application ne propose aucune suppression métier. Les migrations n' emploient pas ON DELETE CASCADE pour les roots ni leurs historiques. Une purge de test est exécutée par transaction de test/troncature d' environnement, jamais par une commande métier.

15. Repositories et Unit of Work

Interface	Responsabilité exclusive
ConsultationRepository	Charge/sauvegarde une Consultation et ses lots/tranches internes, filtrés tenant.
DceVersionRepository	Charge/sauvegarde une DceVersion et enfants documentaires, filtrés tenant.
CaseRepository	Charge/sauvegarde Case et ses historiques internes, filtrés tenant.

DecisionRepository	Charge/sauvegarde Decision, contextes et conditions internes, filtrés tenant.
CommandReceiptRepository	Réserve/rejoue les intentions idempotentes ; ne connaît pas la logique métier.
EventOutboxRepository	Ajoute événements/outbox dans la transaction courante ; ne publie pas.
ProcessInboxRepository	Déduplique l' exécution d' un Process Manager.

Aucun repository n' importe le modèle ORM d' un autre root pour le muter. Le handler peut demander à une interface de lecture une référence versionnée, mais l' écriture reste exclusivement celle de son root propriétaire.

Partie VI — Plan de migrations Alembic

16. Arborescence à créer

Plain Text

```
alembic/
├─ env.py
├─ script.py.mako
└─ versions/
    ├─ 20260813_0001_platform_command_durability.py
    ├─ 20260813_0002_consultation_dce.py
    ├─ 20260813_0003_case.py
    └─ 20260813_0004_decision.py
```

Les migrations sont écrites et relues manuellement. `alembic revision --autogenerate` peut proposer un diff, mais il ne devient jamais un script appliqué sans vérification humaine : les index partiels, FKs composites, triggers et contraintes métier doivent être présents explicitement.

17. Migration 20260813_0001_platform_command_durability

Objet créé	Contenu
------------	---------


```

        ["status", "lease_expires_at"],
    )

def downgrade() -> None:
    op.drop_table("command_receipts")
    # Les autres tables de la migration sont supprimées en ordre inverse des FK.

```

Le script réel complète les colonnes listées aux sections 5 et 6 avant sa première application.

18. Migration 20260813_0002_consultation_dce

Ordre d' upgrade	Objet
1	consultations , contraintes d' identité et index partiels.
2	consultation_lots , consultation_tranches .
3	dce_versions avec self-FK de prédécesseur.
4	dce_documents , classifications, issues, déclarations de manque, source statements.
5	Triggers protect_admitted_dce_content() et protection de document.
6	Index d' exploitation DCE et tests migration à blanc.

down_revision = "20260813_0001" . Le downgrade est autorisé sur environnement vide de développement uniquement. Toute base contenant des données métier devra être restaurée plutôt que downgradée pour éviter de détruire des corpus documentaires.

19. Migration 20260813_0003_case

Ordre d' upgrade	Objet
1	Table cases avec FKs tenant-scoped vers consultations et dce_versions .
2	Index unique partiel de functional_identity_hash pour les Cases non archivées.

3	Tables <code>case_consultation_links</code> et <code>case_dce_applicability_history</code> .
4	Checks des axes lifecycle, stage, readiness, freshness et scope.
5	Indices portefeuille/rectificatif.

`down_revision = "20260813_0002"` . Le script n' ajoute aucune FK vers Decision, Pricing, Task ou Submission.

20. Migration `20260813_0004_decision`

Ordre d' upgrade	Objet
1	Table <code>decisions</code> , FKs tenant-scoped Case/successor, checks des axes et finalisation.
2	<code>decision_contexts</code> et index unique partiel de context final sélectionné.
3	<code>decision_context_references</code> et <code>decision_conditions</code> .
4	Trigger <code>protect_frozen_decision_context()</code> .
5	Indices de dossier décision et décisions à revoir.

`down_revision = "20260813_0003"` . Les commandes de finalisation ne sont activées qu' après cette migration et les tests `DEC-*` , `CONC-02` , `DB-07/08` verts.

Partie VII — Tests de schéma et critères de sortie DATA-01

21. Correspondance DATA-01 → TEST-01

Élément DATA-01	Tests TEST-01 obligatoires
-----------------	----------------------------

Tenant sur chaque table + FKs composites	DB-02 , SEC-01 , SEC-05 .
command_receipts et unicités	APP-02..08 , DB-03/04 , CONC-03/07 , API-06/07 .
Révision root / update guard	APP-09/10 , DB-09 , CONC-01/02 .
Root + event + outbox + receipt	APP-01 , DB-07/08 , PROC-07 .
Consultation functional identity	CONS-01/02 , DB-01/02 .
DCE immutable + supersession	DCE-01..11 , DB-05/06 , CONC-05 , PROC-04 .
Case active identity / références	CASE-01..19 , CONC-04 , ARCH-01/02/05/08 .
Decision context/condition/finalization	DEC-01..14 , CONC-02/05 , SEC-03 .
ORM boundaries	ARCH-01..10 , spécialement ARCH-05 .
Process inbox/outbox dedupe	PROC-01..07 .

22. Contrôles obligatoires après chaque migration

Contrôle	Commande / preuve attendue
Base vide	alembic upgrade head réussit sur PostgreSQL neuf.
Vérification metadata	alembic check ne révèle pas de diff inattendu après import des modèles.
Contraintes	Tests DB TEST-01 verts sur base éphémère.
Migration répétable	Création puis destruction de base locale et upgrade head répétés sans état manuel.
FKs et cascades	Inspection metadata + test ARCH-05 verts.
Downgrade dev	Upgrade/downgrade d' une base vide seulement, selon scripts documentés.
Confidentialité	response_body_json , payload_json et schémas du slice ne contiennent pas de champs financiers/secrets.

23. Décisions de gel DATA-01

1. Le premier slice utilise PostgreSQL comme source transactionnelle unique et SQLAlchemy/Alembic comme outils de mapping/migration ; aucun cache ne détient un état métier propriétaire.
 2. Chaque root porte `tenant_id` et `aggregate_revision` ; toute table cible de FK composite porte `UNIQUE (tenant_id, id)` . Toute FK de domaine entre roots est tenant-scoped par clé composite ou validée explicitement avant écriture lorsqu' elle est polymorphe.
 3. `Case` référence `Consultation` et `DceVersion` , mais n' en possède aucune table enfant ni relation ORM mutable.
 4. `DceVersion` possède corpus, documents, classifications, issues et `SourceStatement` ; son corpus est protégé contre la réécriture et remplacé uniquement par une nouvelle version.
 5. `Decision` possède contextes et conditions ; un contexte figé final ne change jamais de contenu.
 6. Les transactions d' écriture incluent root, événement, outbox et résultat idempotent, mais jamais un second root métier.
 7. Aucun `ON DELETE CASCADE` inter-aggregate, aucune relation SQLAlchemy cross-root avec `delete-orphan` , aucun handler/repository multi-root.
 8. Les migrations Alembic sont manuelles, séquencées et validées par TEST-01 avant la première base de développement partagée.
 9. Toute évolution de table doit partir d' une commande, d' un invariant, d' un événement ou d' un test identifié ; aucune colonne « au cas où » n' est ajoutée.
-

Références internes

- `SMART_AO_V8_DOMAIN_01_AGGREGATE_OWNERSHIP_MATRIX.md` — DOMAIN-01 v1.1.
 - `SMART_AO_V8_DOMAIN_03_STATE_MACHINES_INVARIANTS_FIRST_SLICE.md` — DOMAIN-03 v1.0.
 - `SMART_AO_V8_APP_01_CONTRATS_PYDANTIC_PREMIER_SLICE.md` — APP-01 v1.0.
 - `SMART_AO_V8_TEST_01_PLAN_TESTS_PREMIER_SLICE.md` — TEST-01 v1.0.
 - `SMART_AO_V8_ARCHITECTURE_INFRASTRUCTURE_REFERENCE.md` — environnement et stack de référence.
-

Fin de DATA-01 — Mapping de persistance et migrations du premier slice — version 1.0